JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FutureTask<V>

java.lang.Object
    java.util.concurrent.FutureTask<V>

Type Parameters:

`V` - The result type returned by this FutureTask's `get` methods

All Implemented Interfaces:

`Runnable`, `Future<V>`, `RunnableFuture<V>`

```
public class FutureTask<V>
extends Object
implements RunnableFuture<V>
```

A cancellable asynchronous computation. This class provides a base implementation of `Future`, with methods to start and cancel a computation, query to see if the computation is complete, and retrieve the result of the computation. The result can only be retrieved when the computation has completed; the `get` methods will block if the computation has not yet completed. Once the computation has completed, the computation cannot be restarted or cancelled (unless the computation is invoked using `runAndReset()`).

A `FutureTask` can be used to wrap a `Callable` or `Runnable` object. Because `FutureTask` implements `Runnable`, a `FutureTask` can be submitted to an `Executor` for execution.

In addition to serving as a standalone class, this class provides `protected` functionality that may be useful when creating customized task classes.

Since:

1.5

Nested Class Summary

Nested classes/interfaces declared in interface Future
`Future.State`

Constructor Summary

Constructors

Constructor	Description
	Creates a `FutureTask` that will, upon running, execute the given `Runnable`,

`FutureTask(Runnable runnable, V result)`	and arrange that `get` will return the given result on successful completion.
`FutureTask(Callable<V> callable)`	Creates a `FutureTask` that will, upon running, execute the given `Callable`.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`boolean`	`cancel (boolean mayInterruptIfR`	Attempts to cancel execution of this task.
`protected void`	`done()`	Protected method invoked when this task transitions to state `isDone` (whether normally or via cancellation).
`Throwable`	`exceptionNow()`	Returns the exception thrown by the task, without waiting.
`V`	`get()`	Waits if necessary for the computation to complete, and then retrieves its result.
`V`	`get(long timeout, TimeUnit unit)`	Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.
`boolean`	`isCancelled()`	Returns `true` if this task was cancelled before it completed normally.
`boolean`	`isDone()`	

		Returns `true` if this task completed.
`V`	**`resultNow`**`()`	Returns the computed result, without waiting.
`void`	**`run`**`()`	Sets this Future to the result of its computation unless it has been cancelled (or has already been invoked, in which case effects are undefined).
`protected boolean`	**`runAndReset`**`()`	Executes the computation without setting its result, and then resets this future to initial state, failing to do so if the computation encounters an exception or is cancelled.
`protected void`	**`set`**`(V v)`	Sets the result of this future to the given value unless this future has already been set or has been cancelled.
`protected void`	**`setException`**`(Throwable t)`	Causes this future to report an `ExecutionException` with the given throwable as its cause, unless this future has already been set or has been cancelled.
`Future.State`	**`state`**`()`	Returns the computation state.
`String`	**`toString`**`()`	Returns a string representation of this FutureTask.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

FutureTask

```
public FutureTask(Callable<V> callable)
```

Creates a `FutureTask` that will, upon running, execute the given `Callable`.

Parameters:

`callable` - the callable task

Throws:

`NullPointerException` - if the callable is null

FutureTask

```
public FutureTask(Runnable runnable,
                  V result)
```

Creates a `FutureTask` that will, upon running, execute the given `Runnable`, and arrange that `get` will return the given result on successful completion.

Parameters:

`runnable` - the runnable task

`result` - the result to return on successful completion. If you don't need a particular result, consider using constructions of the form: `Future<?> f = new FutureTask<Void>(runnable, null)`

Throws:

`NullPointerException` - if the runnable is null

Method Details

isCancelled

```
public boolean isCancelled()
```

Description copied from interface: `Future`

Returns `true` if this task was cancelled before it completed normally.

Specified by:

`isCancelled` in interface `Future<V>`

Returns:

`true` if this task was cancelled before it completed

isDone

```
public boolean isDone()
```

Description copied from interface: `Future`

Returns `true` if this task completed. Completion may be due to normal termination, an exception, or cancellation -- in all of these cases, this method will return `true`.

Specified by:

`isDone` in interface `Future<V>`

Returns:

`true` if this task completed

cancel

```
public boolean cancel(boolean mayInterruptIfRunning)
```

Description copied from interface: `Future`

Attempts to cancel execution of this task. This method has no effect if the task is already completed or cancelled, or could not be cancelled for some other reason. Otherwise, if this task has not started when `cancel` is called, this task should never run. If the task has already started, then the `mayInterruptIfRunning` parameter determines whether the thread executing this task (when known by the implementation) is interrupted in an attempt to stop the task.

The return value from this method does not necessarily indicate whether the task is now cancelled; use `Future.isCancelled()`.

Specified by:

`cancel` in interface `Future<V>`

Parameters:

`mayInterruptIfRunning` - `true` if the thread executing this task should be interrupted (if the thread is known to the implementation); otherwise, in-progress tasks are allowed to complete

Returns:

`false` if the task could not be cancelled, typically because it has already completed; `true` otherwise. If two or more threads cause a task to be cancelled, then at least one of them returns `true`. Implementations may provide stronger guarantees.

get

```
public V get()
      throws InterruptedException,
             ExecutionException
```

Description copied from interface: Future

Waits if necessary for the computation to complete, and then retrieves its result.

Specified by:

get in interface Future<V>

Returns:

the computed result

Throws:

CancellationException - if the computation was cancelled

InterruptedException - if the current thread was interrupted while waiting

ExecutionException - if the computation threw an exception

get

```
public V get(long timeout,
             TimeUnit unit)
      throws InterruptedException,
             ExecutionException,
             TimeoutException
```

Description copied from interface: Future

Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

Specified by:

get in interface Future<V>

Parameters:

timeout - the maximum time to wait

unit - the time unit of the timeout argument

Returns:

the computed result

Throws:

CancellationException - if the computation was cancelled

InterruptedException - if the current thread was interrupted while waiting

ExecutionException - if the computation threw an exception

TimeoutException - if the wait timed out

resultNow

```
public V resultNow()
```

Description copied from interface: Future

Returns the computed result, without waiting.

This method is for cases where the caller knows that the task has already completed successfully, for example when filtering a stream of Future objects for the successful tasks and using a mapping operation to obtain a stream of results.

```
    results = futures.stream()
                .filter(f -> f.state() == Future.State.SUCCESS)
                .map(Future::resultNow)
                .toList();
```

Specified by:

resultNow in interface Future<V>

Returns:

the computed result

Since:

19

exceptionNow

```
public Throwable exceptionNow()
```

Description copied from interface: Future

Returns the exception thrown by the task, without waiting.

This method is for cases where the caller knows that the task has already completed with an exception.

Specified by:

exceptionNow in interface Future<V>

Returns:

the exception thrown by the task

Since:

19

state

```
public Future.State state()
```

Description copied from interface: `Future`

Returns the computation state.

Specified by:

`state` in interface `Future<V>`

Returns:

the computation state

Since:

19

done

```
protected void done()
```

Protected method invoked when this task transitions to state `isDone` (whether normally or via cancellation). The default implementation does nothing. Subclasses may override this method to invoke completion callbacks or perform bookkeeping. Note that you can query status inside the implementation of this method to determine whether this task has been cancelled.

set

```
protected void set(V v)
```

Sets the result of this future to the given value unless this future has already been set or has been cancelled.

This method is invoked internally by the `run()` method upon successful completion of the computation. Invocation in other contexts has undefined effects. Any override of this method in subclasses should include `super.set(v)`.

Parameters:

`v` - the value

setException

```
protected void setException(Throwable t)
```

Causes this future to report an `ExecutionException` with the given throwable as its cause, unless this future has already been set or has been cancelled.

This method is invoked internally by the `run()` method upon failure of the computation.

Parameters:

`t` - the cause of failure

run

```
public void run()
```

Description copied from interface: `RunnableFuture`

Sets this Future to the result of its computation unless it has been cancelled (or has already been invoked, in which case effects are undefined).

Specified by:

`run` in interface `Runnable`

Specified by:

`run` in interface `RunnableFuture<V>`

runAndReset

```
protected boolean runAndReset()
```

Executes the computation without setting its result, and then resets this future to initial state, failing to do so if the computation encounters an exception or is cancelled. This is designed for use with tasks that intrinsically execute more than once.

Returns:

`true` if successfully run and reset

toString

```
public String toString()
```

Returns a string representation of this FutureTask.

Overrides:

`toString` in class `Object`

Implementation Requirements:

The default implementation returns a string identifying this FutureTask, as well as its completion state. The state, in brackets, contains one of the strings `"Completed Normally"`, `"Completed Exceptionally"`, `"Cancelled"`, or `"Not completed"`.

Returns:

a string representation of this FutureTask

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.